



Chapter 6.3

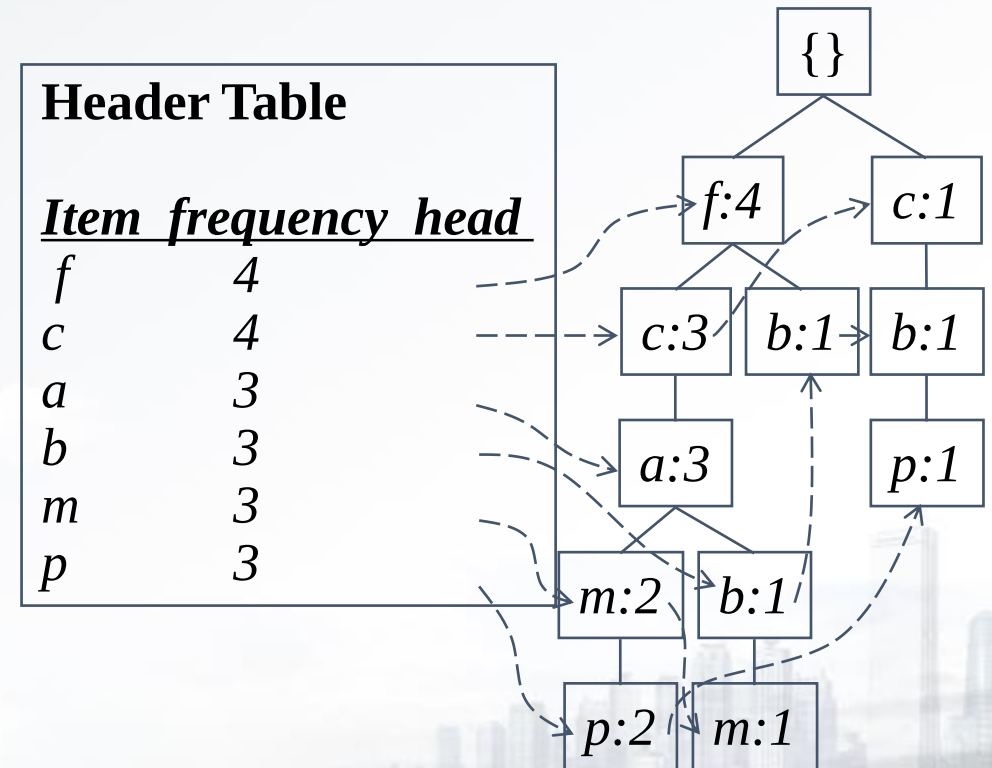
FP-growth算法

Apriori算法的优缺点

- 优点
 - 算法原理简单，易于理解。
- 缺点：
 - 需要多次扫描数据集
 - 如果频繁项集最多包含10个项，需要扫描事务数据集10次，这需要很大的I/O负载
 - 产生大量频繁项集
 - 如数据集有100项，可能产生的候选项个数为 1.27×10^{30}

频繁模式增长(frequent-pattern growth FP-Growth)

- 将提供频繁项集的数据库压缩到FP-树，但仍保持项集关联信息;
- 压缩后的数据库分成一组条件数据库，每个数据库关联一个频繁项，并分别挖掘每个条件数据库;



FP-growth算法实现步骤

- ①第一次扫描事务数据集D，确定每个1项集的支持度计数，将频繁1项集按照支持度计数降序排序，得到排序后的频繁1项集集合L。
- ②第二次扫描事务数据集D，读出每个事务并构建根结点为null的FP-tree。
 - i. 创建FP-tree的根结点，用null标记；
 - ii. 将事务数据集D中的每个事务中的集，删除非频繁项，将频繁项按照L中的顺序重新排列事务中项的顺序，并对每个事务创建一个分支；
 - iii. 当为一个事务考虑增加分支时，沿共同前缀上的每个结点的计数加1，为跟随前缀后的项创建结点并连接；
 - iv. 创建一个项头表，以方便遍历，每个项通过一个结点链指向它在树中的出现。

FP-growth算法实现步骤

- ③从1项集的频繁项集中支持度最低的项开始，从项头表的结点链头指针，沿循每个频繁项的链接来遍历FP-tree，找出该频繁项的所有前缀路径，构造该频繁项的条件模式基，并计算这些条件模式基中每一项的支持度；
- ④通过条件模式基构造条件FP-tree，删除其中支持度低于最小支持度阈值的部分，满足最小支持度阈值的部分则是频繁项集；
- ⑤递归地挖掘每个条件FP-tree，直到找到FP-tree为空或者FP-tree只有一条路径，该路径上的所有项的组合都是频繁项集。

测试数据集

TID	Items
1	面包、可乐、麦片
2	牛奶、可乐
3	牛奶、面包、麦片
4	牛奶、可乐
5	面包、鸡蛋、麦片
6	牛奶、面包、可乐
7	牛奶、面包、鸡蛋、麦片
8	牛奶、面包、可乐
9	面包、可乐

例6.8 FP-growth算法

该数据集具有9个事务，设最小支持度为2，频繁项集的极大长度为3。试使用FP-growth算法挖掘表6-2的事务数据中的频繁项集。

表 6-4 更新后的某商店的详细事务数据

TID	Items	Sorted Items
1	面包, 可乐, 麦片	面包, 可乐, 麦片
2	牛奶, 可乐	牛奶, 可乐
3	牛奶, 面包, 麦片	面包, 牛奶, 麦片
4	牛奶, 可乐	牛奶, 可乐
5	面包, 鸡蛋, 麦片	面包, 麦片, 鸡蛋
6	牛奶, 面包, 可乐	面包, 牛奶, 可乐
7	牛奶, 面包, 鸡蛋, 麦片	面包, 牛奶, 麦片, 鸡蛋
8	牛奶, 面包, 可乐	面包, 牛奶, 可乐
9	面包, 可乐	面包, 可乐

项集	支持度计数	结点链头指针
面包	7	——
牛奶	6	——
可乐	6	——
麦片	4	——
鸡蛋	2	——

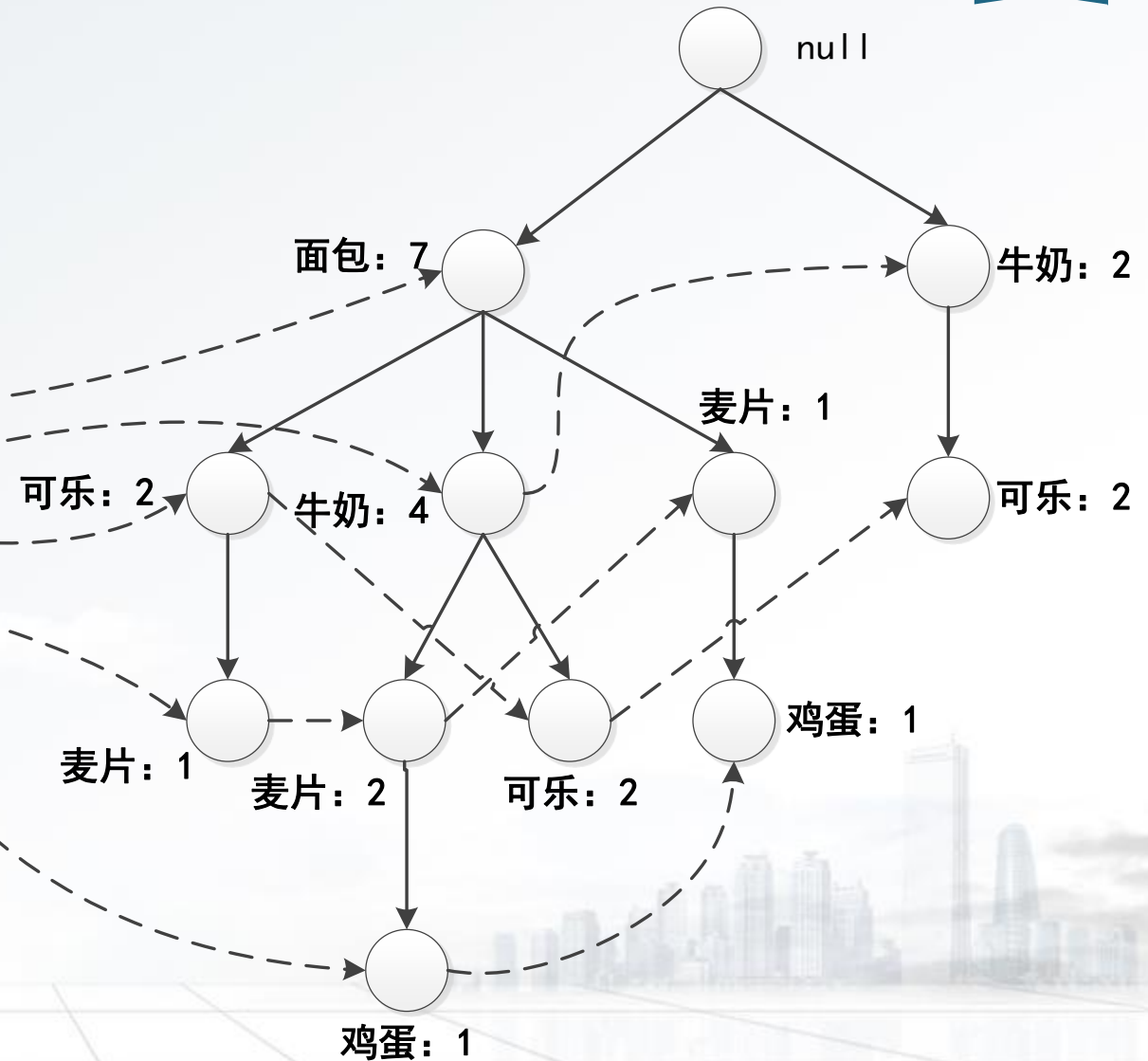


图6-11 FP-tree

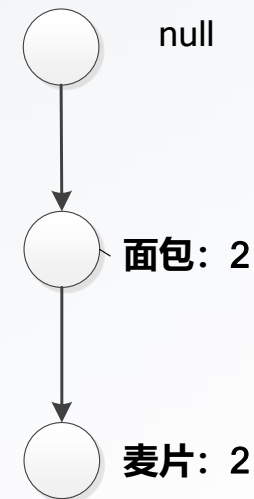
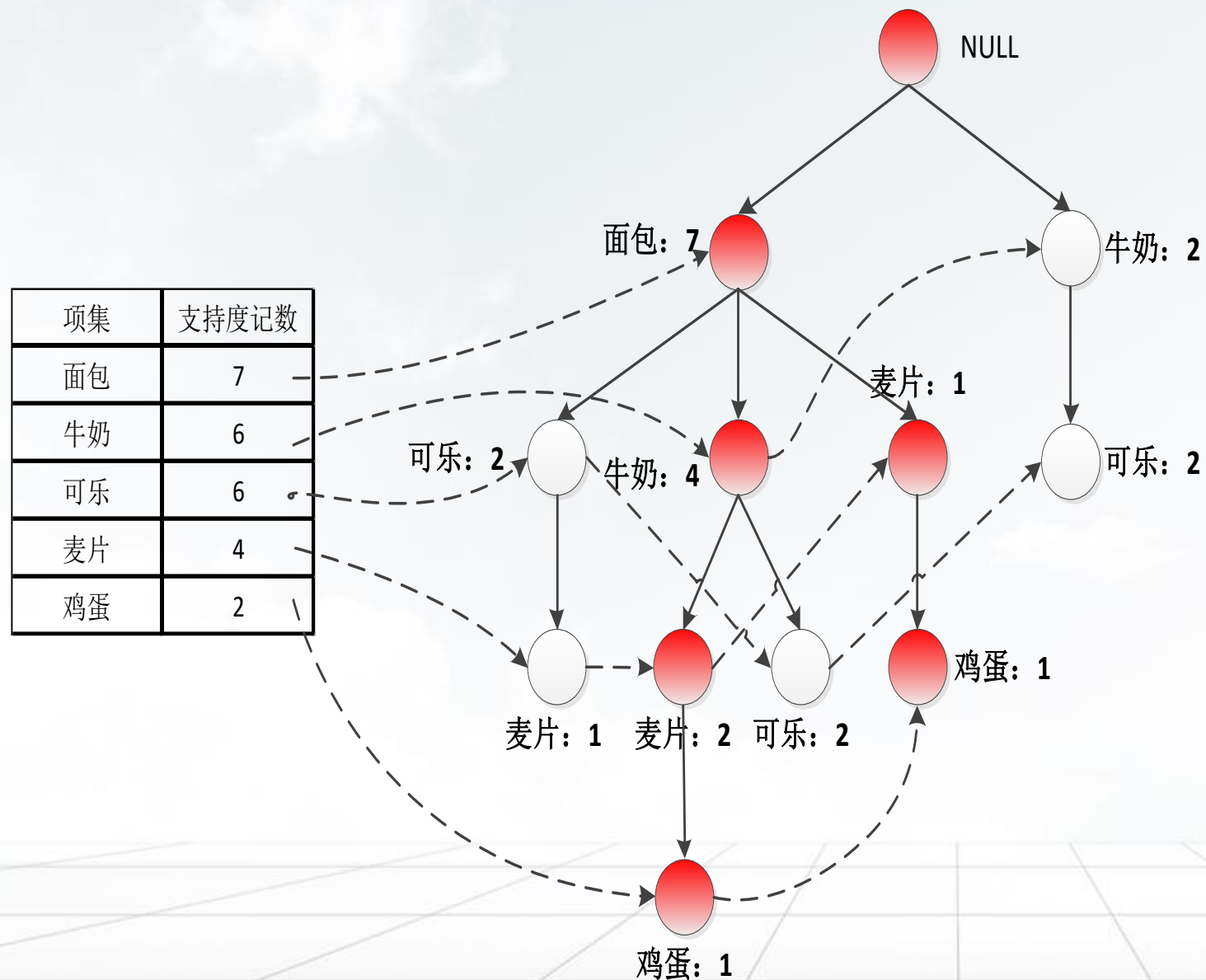


图6-12 {鸡蛋}的条件FP-tree

对于项集{鸡蛋}, 开始向上找出所有前缀路径, 找出其中的频繁模式:

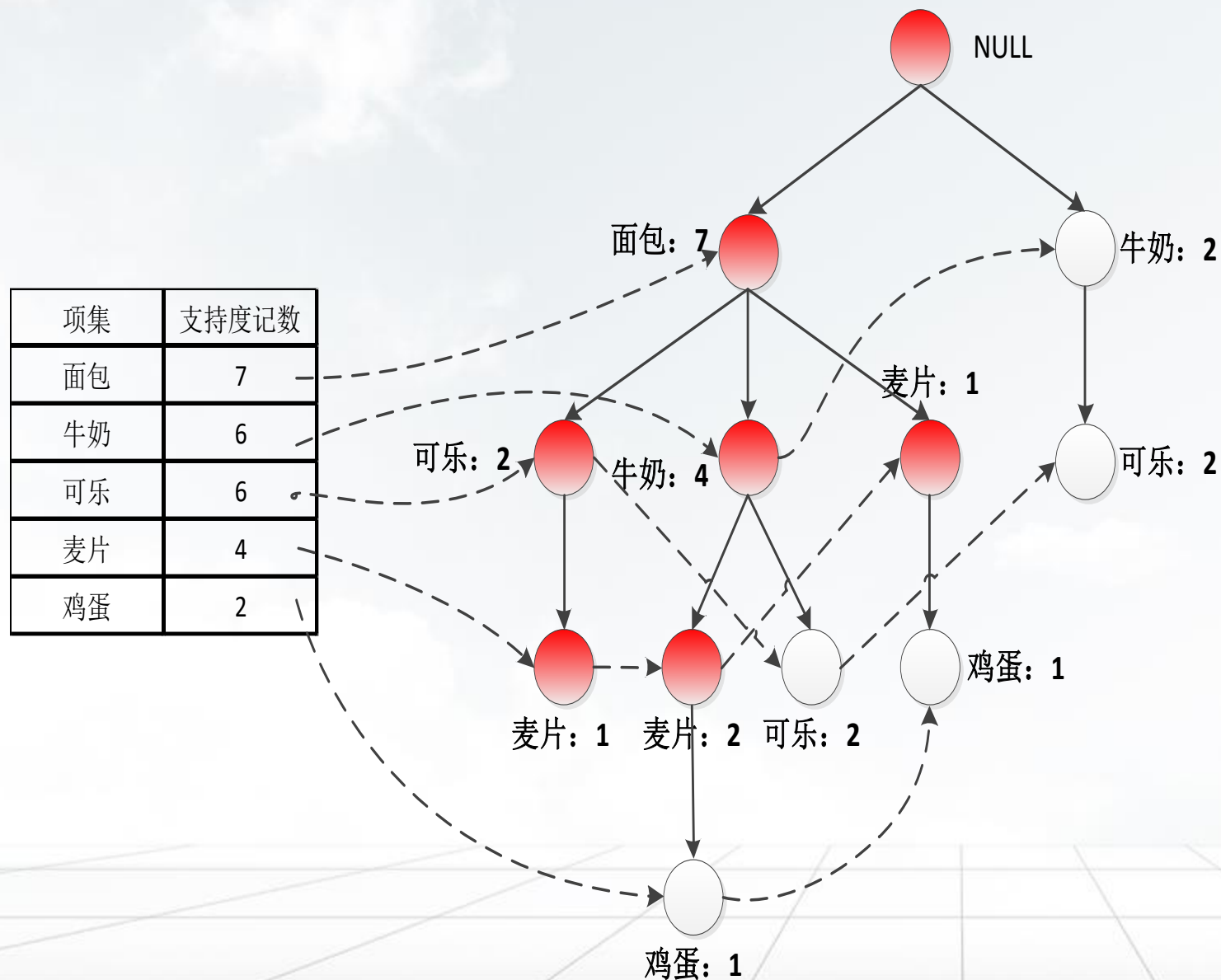
{面包, 鸡蛋: 2}

{面包, 麦片, 鸡蛋: 2}

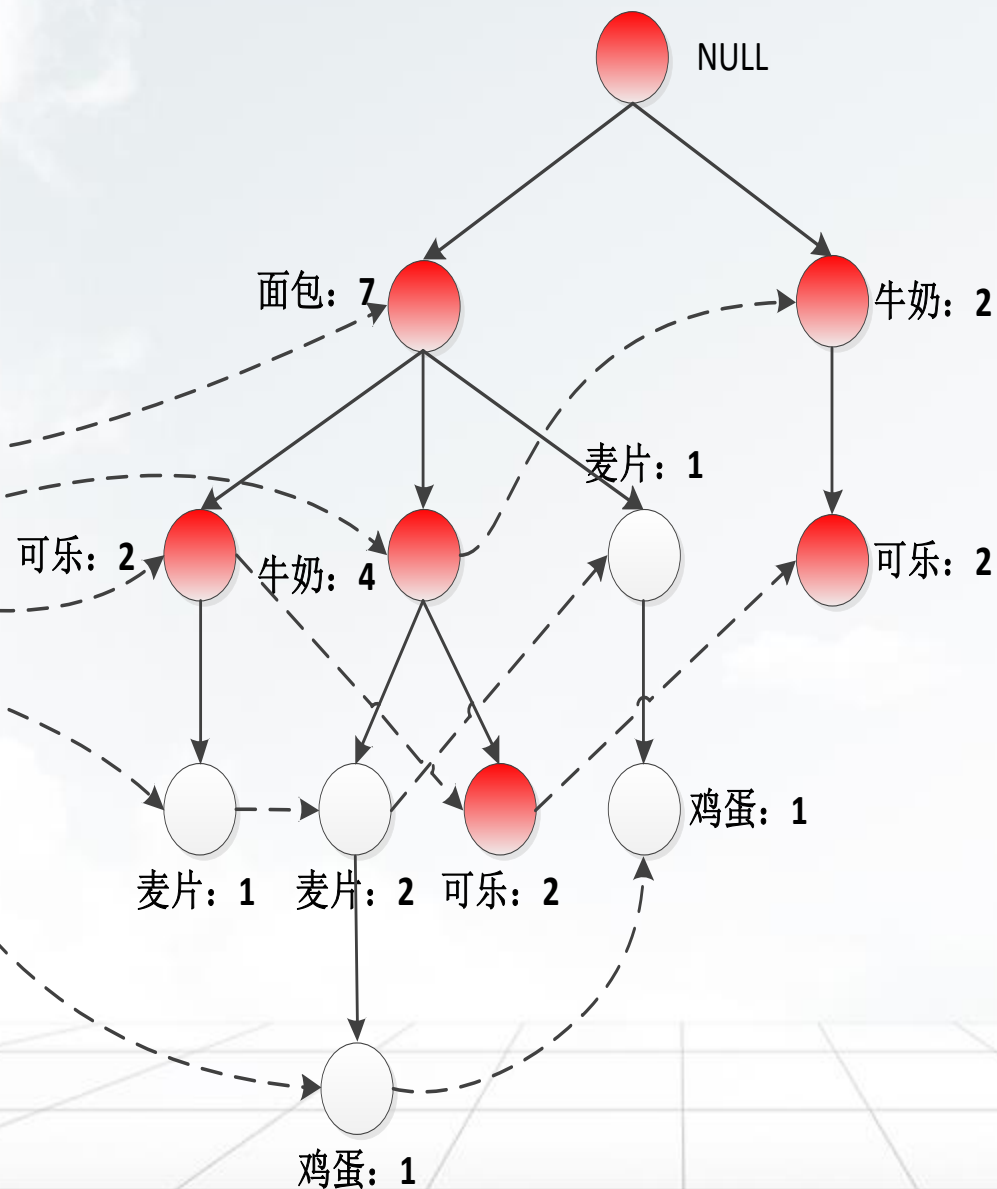
{面包, 麦片, 鸡蛋: 2}

➡ 6.3 FP-growth算法

35



项集	支持度记数
面包	7
牛奶	6
可乐	6
麦片	4
鸡蛋	2

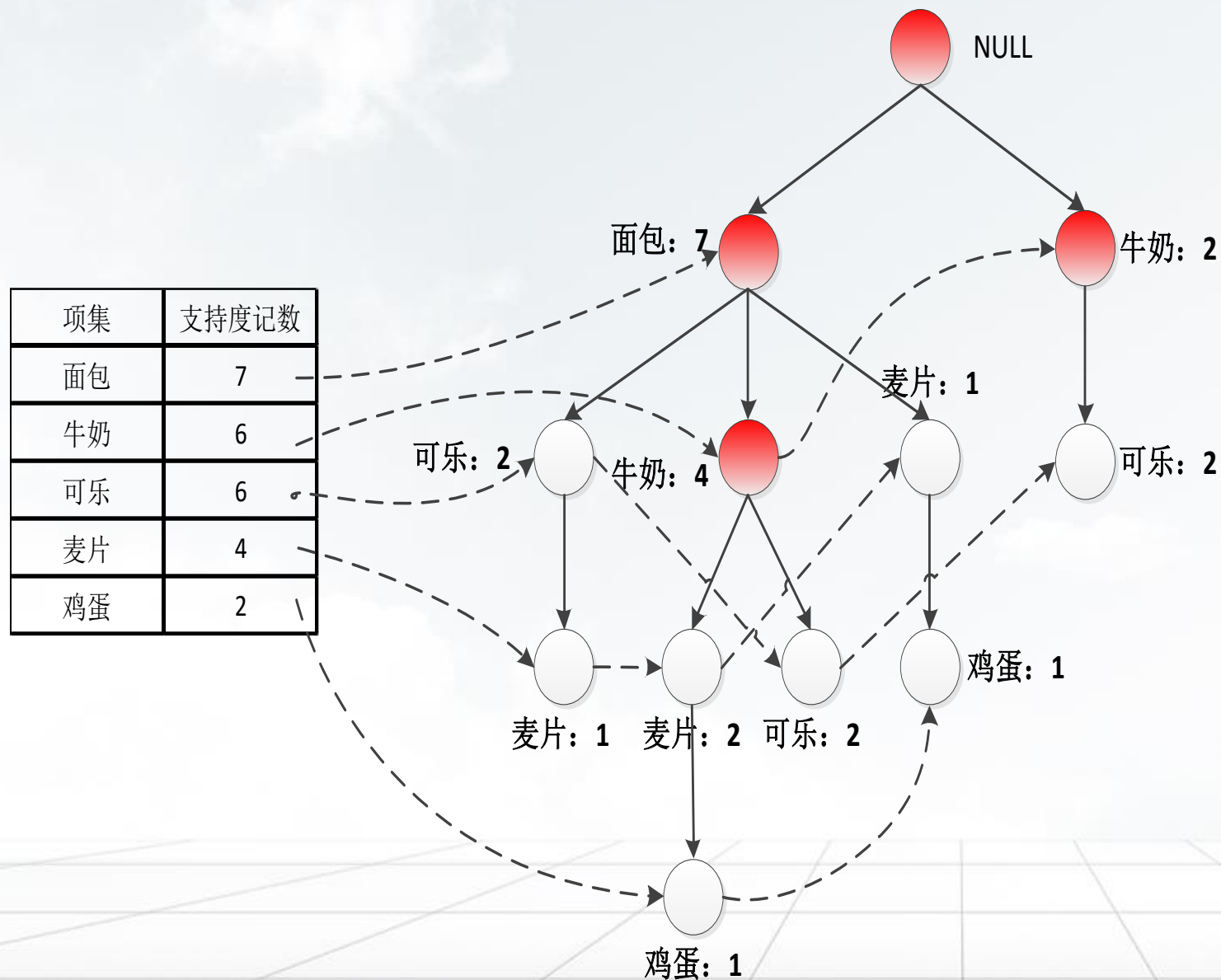


对于项集{可乐}, 开始向上找出所有前缀路径, 找出其中的频繁模式:

- {面包, 可乐: 4}
- {牛奶, 可乐: 4}
- {面包, 牛奶, 可乐: 2}

➡ 6.3 FP-growth算法

37



对于项集{牛奶}, 开始向上找出所有前缀路径, 找出其中的频繁模式:

{面包, 牛奶: 4}

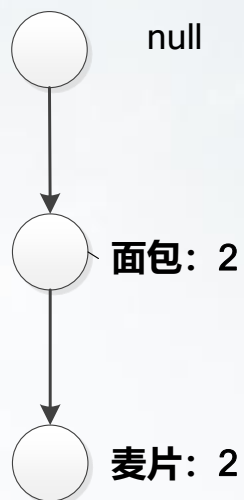


表6-5 FP-tree 挖掘过程

项集	条件模式基	条件FP-tree	频繁模式
鸡蛋	{{面包, 牛奶, 麦片: 1}, {面包, 麦片: 1}}	<面包: 2, 麦片: 2>	{面包, 鸡蛋: 2}, {麦片, 鸡蛋: 2}, {面包, 麦片, 鸡蛋: 2}
麦片	{{面包, 可乐: 1}, {面包, 牛奶: 2}, {面包: 1}}	<面包: 4, 牛奶: 2>	{面包, 麦片: 4}, {牛奶, 麦片: 2}, {面包, 牛奶, 麦片: 2}
可乐	{{面包: 2}, {面包, 牛奶: 2}, {牛奶: 2}}	<面包: 4, 牛奶: 2>, <牛奶: 2>	{面包, 可乐: 4}, {牛奶, 可乐: 4}, {面包, 牛奶, 可乐: 2}
牛奶	{{面包: 4}}	<面包: 4>	{面包, 牛奶: 4}

图6-12 {鸡蛋}的条件FP-tree

— 性能研究表明

- FP-树比Apriori算法快一个数量级

— 原因

- 减少没有候选集的产生，没有候选测试
- 使用简洁的数据结构
- 除去了重复的数据库扫描